

Consulting Engagement Briefing — Blink Oftalmologia × LangChain/LangSmith

Purpose: Request architectural guidance + Solutions Engineering consulting for "Lia", a healthcare WhatsApp agent in production. Includes scope, current pain points, and 5 key technical questions for the LangChain Solutions Architect team.

Date: 05/06/2026 · **Owner:** Fábio Philipe Martins (CEO Blink Oftalmologia)

1. Executive Summary (60 seconds)

Blink Oftalmologia (Brasília-DF, Brazil) runs an ophthalmology clinic with 3 doctors and 2 units. We have a production WhatsApp agent ("Lia") built on **Anthropic Claude Sonnet 4.5 + Haiku 4.5** that handles patient triage, scheduling, and post-consultation flows. The agent integrates **Kommo CRM**, **Medware** (legacy clinic management software), and **WhatsApp Cloud API**.

We've reached a maturity ceiling: 250+ engineering tasks shipped in 15 days, 700+ pytest cases, 13+ regex post-generation filters, FSM state machine, but **recurring bugs persist** in the agenda-presentation flow (model hallucinations, wrong weekday for date, skipping closer slots, presenting slots that don't exist in Medware).

We want a Solutions Architect from LangChain to validate our architecture and recommend whether to migrate to **LangGraph + LangSmith** for: (a) forced tool calling per FSM state, (b) production observability, (c) automatic evaluation on regression bugs, (d) reliable deployment infrastructure.

2. Current Stack

Layer	Today	Pain
Orchestration	Custom Python FSM (fsm_conversa.py) + responder.py calling Anthropic SDK directly	Model NOT invoking tools when state=AGENDA (Bug #183)
Models	Sonnet 4.5 (reasoning) + Haiku 4.5 (judge, currently OFF)	Sonnet diluted attention in 15k-token system prompt
Tools	voice_agent/tools_lia.py with oferecer_slot , confirmar_dados_paciente , gravar_agendamento_medware	Schema OK but model writes free text instead of calling them
Memory	Redis state + Prompt Caching SDK + custom RAG over 38 KB articles	No proper trace/eval pipeline
Hosting	Easypanel (Docker) at blink-agent.6prkfn.easypanel.host	IP blocked by Kommo WAF (Bug #240)
Observability	Custom 5 pillars (leads-fantasma, watchdog, replay, canary, smoke)	Failed to detect lead 24107106 abandoned today (90 min, no alert)
Testing	700+ pytest + GitHub Actions CI	All synthetic; no eval against real conversation transcripts
Defense layers	13+ regex filters post-generation	Each new bug requires new filter — long tail

3. Top 5 Recurring Bugs (production-validated)

Bug	Patient/Case	Symptom
Wrong weekday	Priscila lead 24055629	Lia offered "Friday 06/06" but 06/06 was Saturday
Closer slot skipped	Pedro Miguel 24102510	Patient asked Monday 29/06 — Lia jumped to 02/07
Fabricated availability	Julienne 24053159	Lia invented "office hours 9am-6pm" without checking Medware
Tool example copied	Adelia 24056883	Lia replied with example placeholder text from system prompt
Redundant turn-period question	Carol/Alice 21256807	Asked "morning or afternoon?" while having real slots in context

After each, we added a regex filter. The filters are growing unbounded. We believe migrating presentation-of-agenda to a **structured tool call that the model is forced to invoke** (with `tool_choice` binding to FSM state) is the architecturally correct move.

4. Why LangChain / LangSmith Specifically

We evaluated: - **Anthropic Solutions Engineering** (via support.claude.com) — generic Claude guidance - **OpenAI Solutions** — wrong vendor for our model choice - **Voiceflow / Cresta** — too vertical, not for our DIY/code-heavy team - **LangChain Enterprise Plan** — fits because: - Native `tool_choice` enforcement via LangGraph state machine - LangSmith for traces + evals against real patient conversations - Production deployment with auth + state persistence - Sandboxes for safely testing agent-generated agendamentos before Medware commit - Engineering team training + architectural guidance (per Enterprise plan) - SLA + dedicated engineer access

5. Engagement Scope — what we'd like to discuss

Phase 1 — Architecture Review (4-8 hours)

- Code review of `voice_agent/responder.py`, `tools_lia.py`, `fsm_conversa.py`, `pipeline.py`
- Validate or refute the "consolidated specialist agent" pattern we propose in `VERSAO_COMPLETA_FLUXO_AGENDA_MEDWARE.md`
- Recommend: stay on raw Anthropic SDK + custom FSM **OR** migrate to LangGraph

Phase 2 — LangSmith Pilot (1-2 weeks)

- Instrument Lia with LangSmith tracing (5k traces/month free tier)
- Build eval dataset from 50 real production transcripts (patient permission already in place)
- Create regression evals for each Bug C-NN listed above
- Train internal team on LangSmith Engine for autonomous diagnosis

Phase 3 — Production Migration (optional, 4-6 weeks)

- Migrate scheduling flow to LangGraph specialist agent
- LangSmith Deployment for production hosting (replaces Easypanel current)
- Sandboxes for staging agendamento submissions before Medware

Out of scope (we handle in-house)

- Brazilian Portuguese tone / clinical content / Medware integration code
- WhatsApp Cloud + Kommo CRM business logic
- Brazilian healthcare regulation (CFM, ANS)

6. Five Specific Technical Questions

- 1. Tool-choice enforcement under FSM:** In LangGraph, what's the recommended pattern to force `tool_choice={"type":"tool","name":"X"}` per node, given that nodes have transitions and the same agent must route across states?
- 2. Hierarchical agents:** Is the "router → specialist" pattern (general Sonnet 4.5 → scheduling Haiku 4.5) well-supported in LangGraph, or do you recommend `deepagents` for this?
- 3. LangSmith Eval on real transcripts:** Can LangSmith Engine cluster failure modes from raw production traces (no annotations) and propose prompt/code fixes? Or does it require human-labeled datasets first?
- 4. State persistence across WhatsApp inbound bursts:** Patients send 5 messages in 3 seconds. We currently use Redis lock keyed on `conversation_key` . Does LangGraph Deployment have native debouncing/batching for this?
- 5. Sandbox for medical-data side-effects:** Is LangSmith Sandboxes adequate for "dry-run an agendamento against Medware staging, get response, but DON'T commit until human-in-loop approves"? Or do we need a different pattern?

7. Company Snapshot

Field	Value
Company	Blink Oftalmologia LTDA
Industry	Healthcare — Ophthalmology clinic
HQ	Brasília-DF, Brazil (LATAM)
Size	21-100 employees
Tech team	1 founder-developer (Fábio Philipe) + external contractors
Annual revenue	Multi-million BRL (private)
Patients/month via Lia	~600 active conversations, ~150 new scheduled
Current model spend	Claude Sonnet 4.5 ~\$800/month, Haiku 4.5 ~\$50/month

8. Decision Timeline

- **Now (week of 05/06/2026):** evaluate LangChain Enterprise vs. continue in-house
- **+2 weeks:** sign contract OR formally close engagement
- **+4 weeks:** Phase 1 architecture review delivered
- **+8 weeks:** Phase 2 LangSmith pilot in production

9. Attachments

- RELATORIO_OMBUDSMAN_05-06-2026.pdf — 13 pages, full self-audit including bugs C-01..C-13
 - VERSAO_COMPLETA_FLUXO_AGENDA_MEDWARE.pdf — 15 pages, proposed implementation from 5 internal engineers
 - GitHub repo: <https://github.com/oabphi-bliip/agente-blink> (private; access granted upon NDA)
 - Production health: <https://blink-agent.6prkfn.easypanel.host/admin/healthz>
-

Contact for follow-up

- **Fábio Philipe Martins** — Founder/CEO Blink Oftalmologia — diretoria@blinkoftalmologia.com.br
 - Preferred language: Portuguese or English
 - Time zone: BRT (UTC-3)
 - Availability: Mon-Fri 14:00-18:00 BRT for calls
-

Versão em Português — para uso interno do Fábio

Resumo do que essa consultoria deve trazer:

1. **Code review por engenheiro sênior LangChain** dos 4 arquivos críticos da Lia (`responder.py`, `tools_lia.py`, `fsm_conversa.py`, `pipeline.py`) — eles vão dizer se a arquitetura atual é o caminho correto ou se devemos migrar pra LangGraph
2. **Validação ou veto** das 11 mudanças que propus na "Versão Completa" (tool calling forçado, `criar_agendamento_seguro`, agente dedicado Haiku, etc)
3. **Acesso ao LangSmith** pra ter observability profissional em vez dos meus pilares custom (a 5k traces/mês grátis no plano Developer)
4. **Treinamento da equipe interna** (você + qualquer freelancer que você contratar) em padrões LangGraph
5. **Suporte SLA** com Solutions Engineer dedicado quando der pau em produção

Custo estimado: - Plano Developer: **grátis** (5k traces/mês — suficiente pra começar) - Plano Plus: **\$39/seat/mês** — pra equipe começar a usar profissionalmente - Plano Enterprise (com SLA + treinamento + engenheiro dedicado): **negociado, tipicamente \$3-15k/mês**

Próximo passo concreto: 1. Acessa <https://www.langchain.com/contact-sales> 2. Preenche o form com o texto pré-pronto (ver arquivo `EMAIL_LANGCHAIN_COLAR_NO_FORM.md`) 3. Anexa este briefing em PDF 4. Eles respondem em 1-2 dias úteis com agendamento de demo

Alternativa mais barata se Enterprise for caro: - Plano **Plus a \$39/seat/mês** + contratar consultor freelance da **LangChain Partner Network** (lista oficial: <https://www.langchain.com/langchain-partner-network>) — partners cobram \$150-300/h, conseguem fazer code review profissional em 4-8h